Investigating Software Security Testing Challenges in Developer Discussions

Md Erfan · Ahmed Ryan · Md Rayhanur Rahman

The University of Alabama, Tuscaloosa, USA

Software Security Course
Research Presentation · Fall 2026

Presenter

Md Erfan

THE UNIVERSITY OF ALABAMA

College of Engineering

Computer Science

Outline

1 Motivation & Problem

2 Background & Research Gap

3 Research Questions

4 Methodology: Data & Coding

5 Findings: RQ1, RQ2 & RQ3

6 Implications, Limitations & Future Work

Motivation: Security Testing Is More Than Detection

Functional testing

Does the feature work?

Known inputs → expected outputs

Example: a payment form accepts valid information and records an order.

Software security testing

Can an attacker abuse it?

Malicious inputs → unauthorized behavior

Example: injection, broken access control, or a vulnerable dependency.

SAST | DAST | Fuzzing | Dependency scanning | CI/CD checks

Central challenge: selecting, configuring, interpreting, and acting on SST tools in real workflows.

What Do Developers Ask?

1 Tool inconsistency

Snyk web scan reports vulnerabilities, while a Jenkins-integrated scan does not.

2 Finding interpretation

Is a flagged issue a real vulnerability or a scanner false positive?

3 Authentication testing

How can unit tests mock authenticated users, roles, or an external identity provider?

4 Remediation actionability

A scanner names a vulnerable package, but a patched version is unavailable.

The question is often “How do I trust or use this finding?” not merely “Can the scanner detect a bug?”

Key Concepts in Software Security Testing

SAST — **Static application security testing**

Inspects source code or compiled artifacts without executing the application.

DAST — **Dynamic application security testing**

Probes a running application through HTTP requests and observed responses.

Fuzzing — **Automated robustness/security testing**

Supplies varied and unexpected inputs to expose crashes or weakness patterns.

SCA — **Software composition analysis**

Checks third-party packages, versions, dependencies, and known CVEs.

Source: Paper §2; definitions simplified for a software security classroom.

Related Work & Research Gap

SST foundations

Surveys of SAST, DAST, fuzzing, penetration testing, and tool capabilities.

Tools and workflows

Empirical scanner studies examine false positives, adoption barriers, configuration, and CI/CD failures.

Q&A-based studies

Stack Overflow has been mined for configuration issues, checked-in secrets, code smells, and maintenance.

Research Gap

Limited empirical understanding of how developers themselves describe SST challenges across tools, authentication, integration, findings, and remediation.

Source: Paper §3, Related Work.

Research Questions

RQ1

SST Taxonomy

What challenges do developers face in software security testing?

RQ2

Problem Characteristics

Which challenges are prevalent or difficult, how do they trend, and which co-occur?

RQ3

Taxonomy Stability

Does the taxonomy classify temporally unseen questions without new categories?

Methodology Overview

1 Keyword sources

Academic + practitioner

2 Retrieve

17,743 SO questions

3 Manually filter

582 SST questions

4 Build taxonomy

8 categories · 31 subcategories

5 Characterize

Prevalence, difficulty, trends, pairs

6 Validate

21 held-out recent questions

Source: Paper §4, Approach; Fig. 1 and RQ2/RQ3 methods.

Keyword Identification & Data Collection

Academic literature (K_1)

ICSE · ISSTA · IEEE S&P · USENIX Security

Root terms: security testing, vulnerabilities, static/dynamic analysis, penetration testing, fuzzing.

Practitioner sources (K_2)

Candidate sources discovered using Gemini, ChatGPT, and Perplexity; retained through manual review.

Examples: Project Zero, PortSwigger, OWASP Top 10, Verizon DBIR.

Normalization & retrieval

Group synonyms; remove overly generic terms; independently review keywords; use Stack Exchange Data Explorer for January 2015–August 2025 questions.

Source: Paper §4.1.2–§4.1.4. No quantitative LLM cross-engine agreement threshold was reported.

Dataset Screening & Taxonomy Coding

17,743 Initial questions — **582** SST-relevant questions — **8 / 31** Categories / subcategories

Inclusion: SST focus

Tooling/platforms/standards; practical vulnerability detection during SST; automation in development workflows.

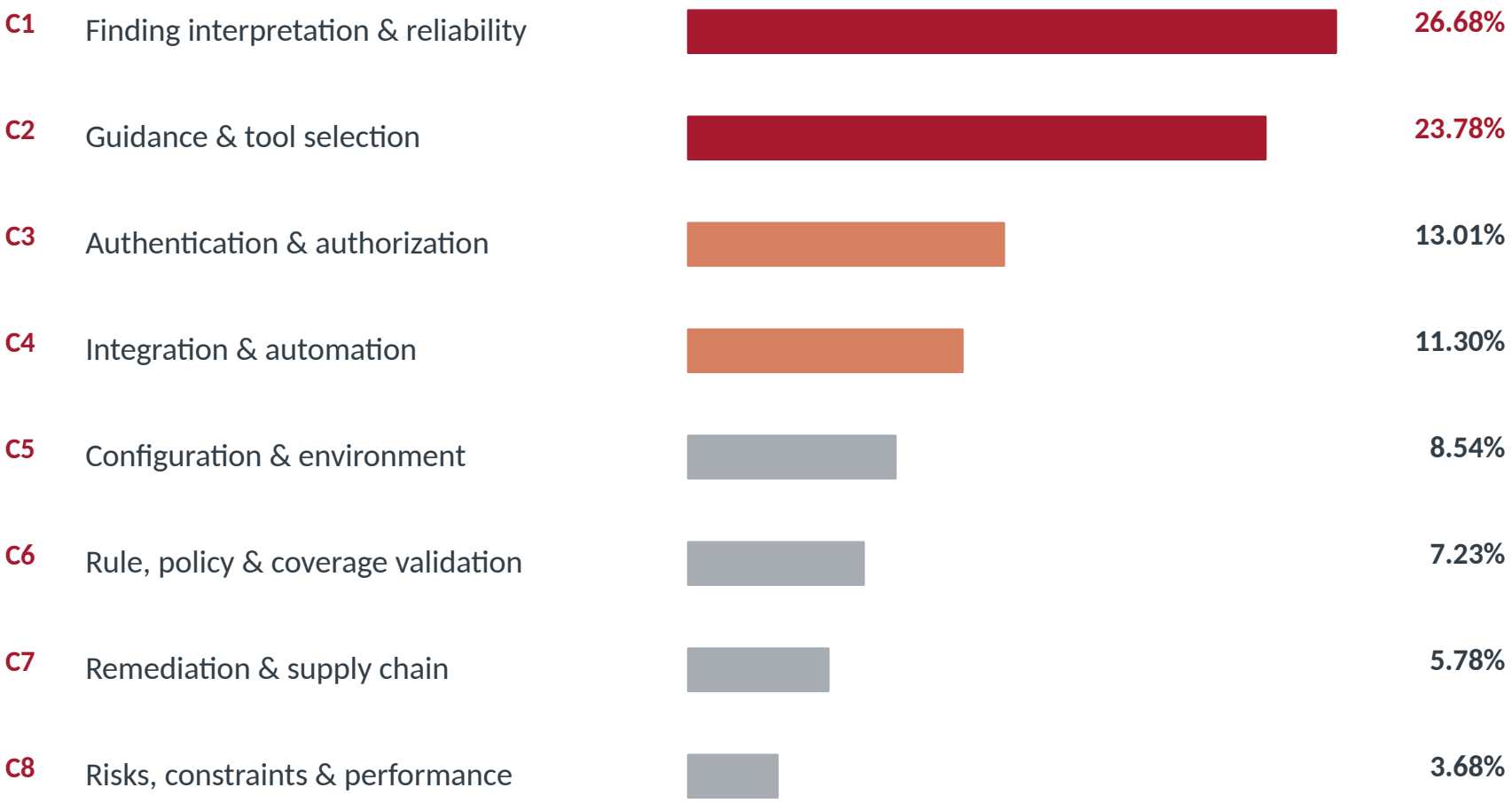
Exclusion: no SST focus

General implementation, ordinary authentication/session management, or routine troubleshooting.

Coding quality: κ improved from 0.61 to 0.69 during relevance labeling; taxonomy labels reached 83% initial raw agreement and consensus resolution.

Source: Paper §4.1.4 and §4.2.

RQ1: Taxonomy of SST Challenges



Source: Paper Fig. 1. Labels, not unique questions, are the denominator because questions can have multiple labels.

RQ1: Security Finding Interpretation & Reliability

77	**False positives**	Is the warning real or a false alarm?
65	**Explainability**	Why did this rule fire?
31	**Inconsistency**	Why do web and CI scans disagree?
20	**Reporting**	Can I export actionable evidence?
10	**False negatives**	Why was a known issue missed?

Takeaway: a scan result is not actionable until the developer can understand and verify it.

RQ1: Authentication & Authorization Testing

55

Mocking

Simulate users, roles, sessions, tokens, and security contexts.

30

Functionality testing

Verify protected endpoints and access-control behavior under realistic conditions.

14

Test setup

Configure credentials, filters, test clients, and identity-provider dependencies.

Highest visibility: 3,803.59 average views/category; mocking alone averaged 4,391.35.

RQ1: Beyond Scanner Output

C2 · Guidance & selection

Tool suggestions, documentation, features, comparative evaluation.

C4 · Integration & automation

Connect scanners to CI/CD, browsers, frameworks, and cloud platforms.

C5 · Configuration & setup

Prepare scan contexts, proxies, runtime conditions, and fuzz inputs.

C6 · Rule/policy/coverage validation

Validate security rules, expected coverage, reproducibility, and mappings.

C7 · Remediation & supply chain

Find dependency path, affected code, and feasible fixes.

C8 · Risks, constraints & performance

Handle secrets, data exposure, lengthy scans, and legal concerns.

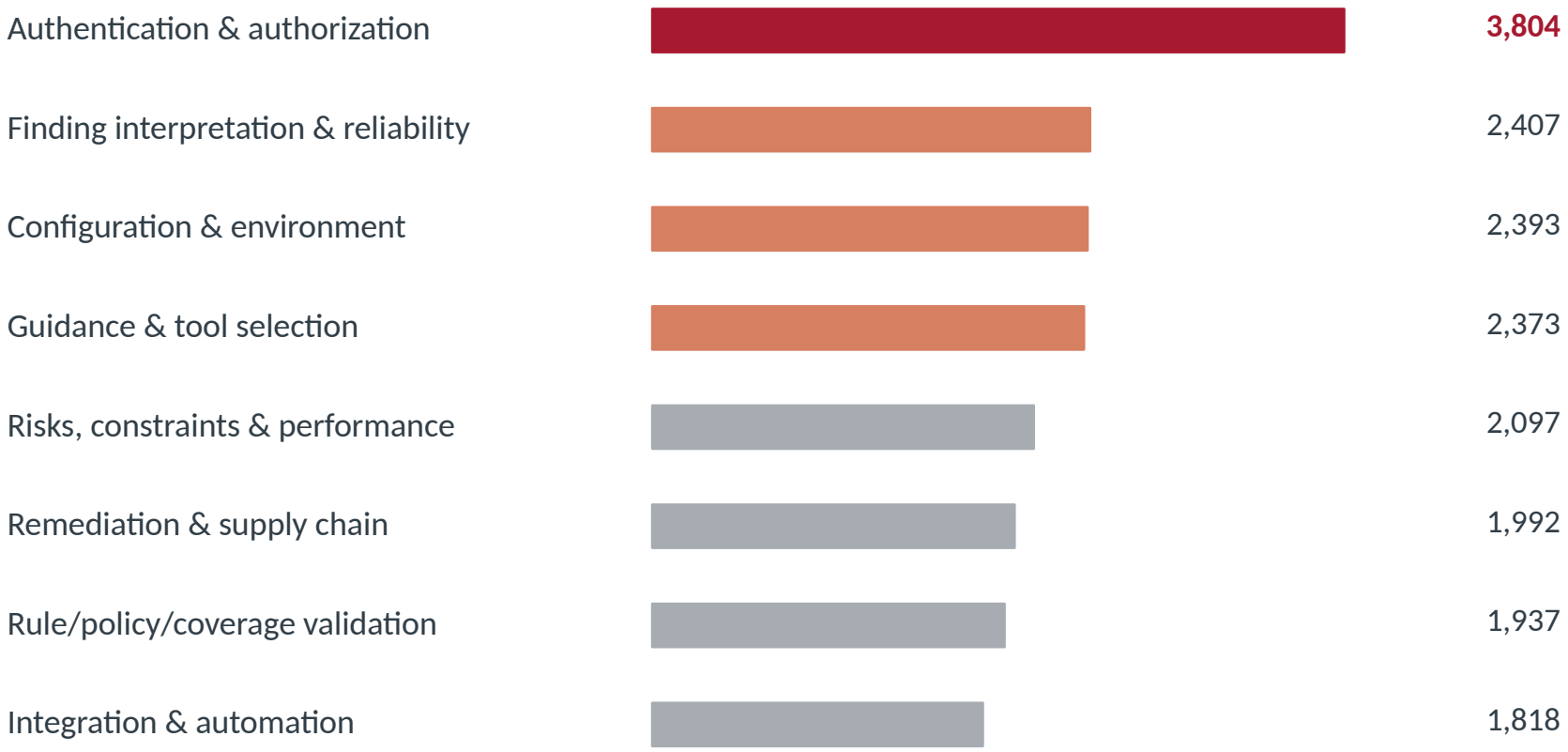
Source: Paper §§5.2 and 5.4–5.8.

RQ2: How Do We Measure Challenge Characteristics?

Prevalence	**Average views · score · answers · comments**	Attention and community engagement
Difficulty	**Answer rate · accepted-answer rate · first/accepted response time · text size**	Resolution and contextual burden
Correlation	**Pearson r and Spearman ρ**	Relationships between prevalence and difficulty metrics
Temporal trend	**Normalized monthly share + Cox–Stuart test**	Increasing, decreasing, or stable category attention
Co-occurrence	**Raw count · Jaccard · lift**	Pairs of challenges in the same question

Source: Paper §4.3. Difficulty reflects SO resolution proxies, not controlled developer-hour measures.

RQ2.1: Which Challenges Attract the Most Attention?



Authentication testing is the most visible category; mocking averages 4,391 views.

RQ2.2: Which SST Challenges Are Hard to Resolve?

41.66 h
Mean accepted-answer time
Rule/policy/coverage

48.45 h
Mean time to first answer
Configuration & setup

0.52
Accepted-answer rate
Authentication & authorization

More context-intensive

Rule/policy/coverage questions had the largest average body length: 2,631.75 characters.

Difficult subcategories

Inconsistency: 0.19 acceptance rate. Test environment readiness: 0.14. Vulnerability localization: 0.11.

Interpretation: question/answer metadata indicates resolution difficulty; it is not a direct measurement of engineering effort.

Source: Paper Table 11; no non-SST syntax-query baseline was measured.

RQ2.3: Relationships Between Difficulty and Engagement

Pearson r **+0.870**

Answer rate ↔ Avg. answers

$p < 0.001$

Spearman ρ **+0.822**

Answer rate ↔ Avg. answers

$p < 0.001$

Pearson r **−0.522**

Accepted-answer rate ↔ Avg. comments

$p < 0.01$

Spearman ρ **−0.467**

Accepted-answer rate ↔ Avg. comments

$p < 0.01$

Takeaway: more discussion and clarification can accompany less conclusively resolved challenges.

RQ2.4: How Do Challenges Evolve Over Time?

↑	**Finding interpretation & reliability**	p = 0.002	**Increasing**
↑	**Remediation & supply chain**	p = 0.006	**Increasing**
↓	**Guidance & tool selection**	p = 0.004	**Decreasing**
↓	**Authentication & authorization**	p = 0.033	**Decreasing**

Consistent: tool integration, configuration/setup, rule validation, and testing risks/performance.

Source: Paper Table 12. Trends describe proportions of SO SST discussions, not vulnerability incidence.

RQ2.5: SST Challenges Often Co-occur

False positives	+	Explainability	14 posts	lift 1.628
Integration	+	Tool suggestions	9 posts	lift 1.645
Integration	+	Documentation / resources	9 posts	lift 1.965
Configuration	+	Fuzzing setup	5 posts	lift 10.981

Interpretation: explainability and false positives connect; successful integration also depends on selection and documentation.

RQ3: Stability on Temporally Unseen Questions

100%

21 of 21 held-out questions covered

No new top-level or subcategory labels required

What was checked?

Could each unseen question's main SST challenge fit one or more existing subcategories?

What does this not prove?

Recent Stack Overflow stability is not independent validation in closed enterprise environments.

4 held-out questions involved multiple SST concerns, supporting multi-label interpretation.

Source: Paper §7; temporal held-out test, 21 unseen questions.

Discussion: What Should Change in SST Tools?

BEFORE

Better selection & configuration

Guided tool choice, reusable setup templates, and authenticated test examples.

DURING

Reliable integration & validation

CI/CD integration, coverage models, consistent results, and security-context handling.

AFTER

Explainable, actionable results

Affected source locations, confidence, dependency paths, reachability, and feasible fixes.

Practical message: security testing is an end-to-end workflow, not a single scanning activity.

Limitations & Threats to Validity

Internal validity

Keyword search may miss informal descriptions. Coding relies on researcher judgment, mitigated through independent labeling, a coding guide, agreement, and consensus.

External validity

Public Stack Overflow posts may underrepresent confidential vulnerability handling, enterprise policy, compliance, private triage, vendor support, and industrial workflows.

The reported frequencies describe this filtered Stack Overflow dataset—not the global prevalence of SST problems.

Conclusions & Future Research

1. A practitioner-derived taxonomy of 8 categories and 31 subcategories from 582 questions.
2. Finding interpretation and tool guidance dominate discussion; authentication testing attracts the most views.
3. Rule/policy/coverage validation is especially challenging; interpretation and remediation attention rise over time.
4. All 21 recent held-out questions fit the existing taxonomy, giving initial stability evidence.

Next steps

Surveys and interviews with developers, security engineers, DevSecOps practitioners and tool builders; replication in GitHub issues, vendor forums, and industrial datasets.

Source: Paper §§8–10, implications, validity, and future work.

Thank You — Questions?

Dataset, code & replication artifacts

figshare.com/s/17e56c96905c8ab423b4

Takeaway

Better SST requires explainable findings, realistic authenticated tests, workflow support, and feasible remediation.

Contact

Md Erfan · Ahmed Ryan · Md Rayhanur Rahman
Department of Computer Science
The University of Alabama

Paper: Investigating Software Security Testing Challenges in Developer Discussions